MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

RISC-V Memory

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- How does RISC-V interact with memory?

Previously

- Memory is byte addressable
- No distinction between data and instructions
- Binary information is binary information

Data Sizes

- 32-bits (word) is the default data type
  - Internally everything is 32-bit
  - Allows processing large numbers in a single step
- But memory is byte addressable? *Why*?
  - Much of our information takes only a few bits
  - A lot of it is characters → bytes
- Processor – ALU mismatch
  - Registers of different sizes?
  - ALUs of different width?

Data Types

	Halfword (h)	Byte (b)
Signed ()	`lh` `sh`	`lb` `sb`
Unsigned (u)	`lhu` `sh`	`lbu` `sb`

- Two other sizes supported for load/store:
  - Halfword → 16-bit, Byte → 8-bit
- Load: `lh, lhu, lb, lbu`
  - Load 16/8 bits into the lower 16/8 bits of the destination register
  - Extend the remaining bits: set to 1 if signed and sign-bit is set, set to 0 otherwise
- Store: `sh, sb`
  - Store the lower 16/8 bits of the source register
  - Sign does not matter

Endianness

- Default size is 32-bit
- Memory is byte (8-bit) addressable
- Words occupy four addresses in memory
  - Which addresses contain which parts of the word?
    - If word is 0x12345678 and
    - We store it in address 0
    - *Which byte is in address 0?*

Endianness

- Two major approaches
- **Little-endian**: address 0 contains the least significant byte of the word
  - If word is 0x12345678 → address 0 contains 0x78
- **Big-endian**: address 0 contains the most significant byte of the word
  - If word is 0x12345678 → address 0 contains 0x12

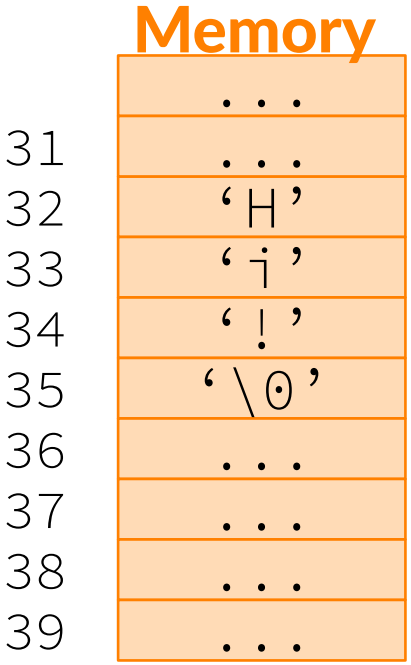
Does this matter?

Endianness: Importance

- Design-wise
  - Lowest addressed bytes the first to process
  - Little-endian useful for ops starting from least significant bytes
    - Addition, Subtraction, Multiplication
  - Big-endian useful for ops starting from most significant bytes
    - Comparison, Division
  - Assignment usually not affected
  - Less important on strictly 32/64-bit processors
    - All bytes loaded at once
    - *But affects software-based extended arithmetic (later this week!)*

Endianness: Importance

- Interpretation-wise
  - Debugging & hex dumps → typically byte based
  - Data packing → multiple byte-sized data in a single word
    - Can happen with strings
    - Little endian: x1 <= [32] → x1 = “\0iH”
    - Big endian: x1 <= [32] → x1 = “Hi!\0”



Endianness

- RISC-V is little endian
  - x1 = 0x12345678
  - sw x1, 12[x0]
- `lw x1, 12[x0]` → `0x12345678`
- `lhu x1, 12[x0]` → `0x00005678`
- `lbu x1, 12[x0]` → `0x00000078`
- Most modern processors are effectively little-endian
- Low-level network communication is big-endian

Memory

	...
11	...
12	0x78
13	0x56
14	0x34
15	0x12
16	...
17	...
18	...
19	...
20	...
21	...
	...

Memory Map

- RISC-V addresses are formed from 32-bit registers
  - So we have 2^{32} (~4 billion) bytes or 2^{30} (~1 billion) words
- This is what can be addressed by RISC-V
  - It doesn't mean that you have all this memory
- How is the memory space used? → Implementation specific
  - Some has actual memory
  - Some is unused
  - Some has hardware devices pretending to be memory (I/O mapped)

Memory Map

- The only architectural limitation is that resetting the processor sets PC to 0
  - Address 0 needs to have *something* that returns a valid instruction
- For now assume a fully populated memory map containing only real memory
  - This is what Bennett simulates
  - For something more real: COMP27112

Summary

- Load/Store halfwords and bytes
- Little-endianness
- 4GB Memory Map
  - Not necessarily 4GBs of memory

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

RISC-V Instruction Encoding

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- How are instructions represented in binary?
- Why are they encoded like that?

Instruction Encoding

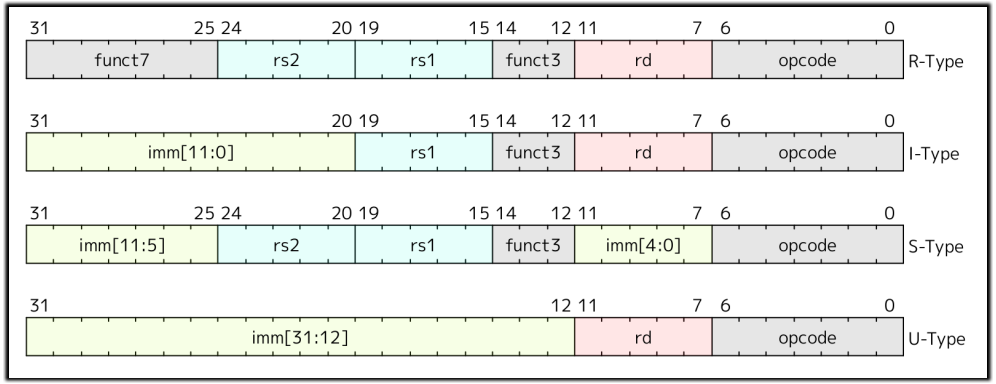
- Assembly
  - A textual representation of the program that is easy to *remember*, *read*, and *write*
  - Not easy for the processor to read
  - e.g., `sub x1, x2, x3` is 15 bytes in ASCII
  - 3.75x times more storage that a 32-bit instruction
  - Complex interpretation (that needs to happen within a fraction of a processor cycle!)

Instruction Encoding

- The actual program is a sequence of bytes
  - Each instruction is 32-bits of machine code
  - In memory each instruction occupies one word
  - How do we go from this
    - `sub x1, x2, x3`
  - to this
    - $403100B3_{16}$
  - in a way that allows the processor to decode this number in picoseconds

Instruction Encoding

- For all instructions
  - Bits[6:0] is the opcode
    - Identifies the instruction type
    - Tells us which encoding to use
    - Can have up to 2^7 = 128 instruction types (Most are reserved)



From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

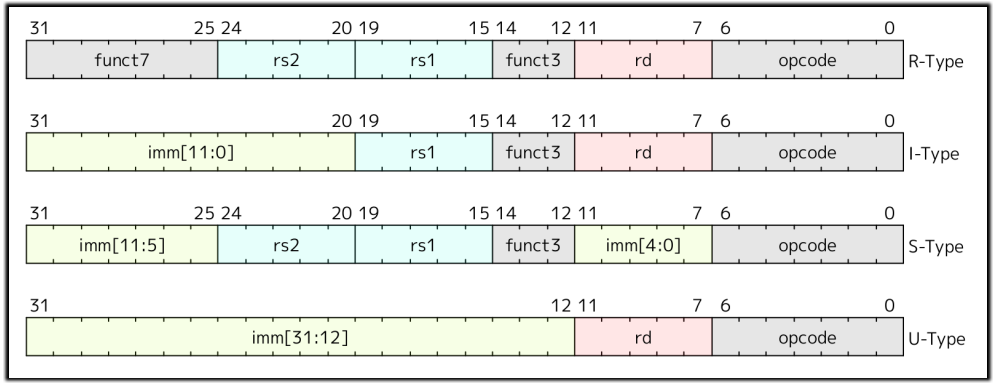
Table 70. RISC-V base opcode map, inst[1:0]=11

inst[4:2] / inst[6:5]	000	001	010	011	100	101	110	111 (>32b)
00	LOAD	LOAD-FP	custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	OP-V	custom-2/rv128	48b
11	BRANCH	JALR	reserved	JAL	SYSTEM	OP-VE	custom-3/rv128	≥80b

From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

Instruction Encoding

- For all instructions
  - All register arguments are 5-bits
    - Can count 2^5 = 32 registers
    - That's how many we have!
    - 17 registers → 5 bits
    - 31 registers → 5 bits
    - 32 registers → 5 bits
    - But 33 registers → 6 bits
  - Same arguments are in the same positions
    - Simpler decoding & it can start before decoding the opcode
  - Most significant bit of the immediate always in bit 31
    - The sign bit: sign extension can start early



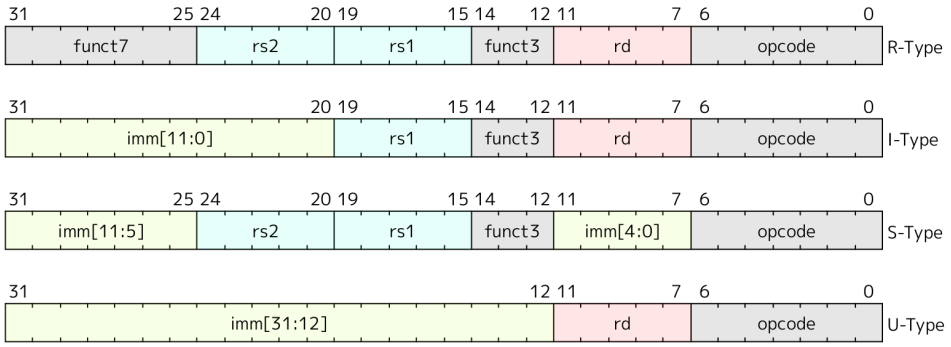
From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

Table 70. RISC-V base opcode map, inst[1:0]=11

inst[4:2] / inst[6:5]	000	001	010	011	100	101	110	111 (>32b)
00	LOAD	LOAD-FP	custom-0	MISC-MEM	OP-IMM	AUIPC	OP-IMM-32	48b
01	STORE	STORE-FP	custom-1	AMO	OP	LUI	OP-32	64b
10	MADD	MSUB	NMSUB	NMADD	OP-FP	OP-V	custom-2/rv128	48b
11	BRANCH	JALR	reserved	JAL	SYSTEM	OP-VE	custom-3/rv128	≥80b

From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

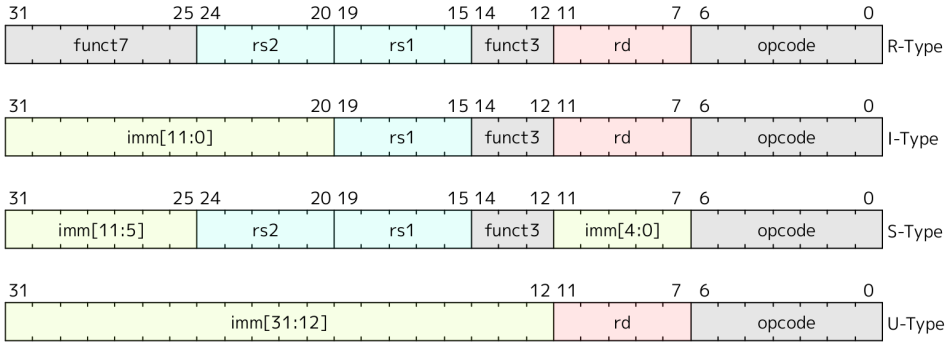
Instruction Encoding



From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

- *R-Type*: Register-Register data processing
  - 2 input and 1 output register operands
  - funct7 + funct3 selects the specific operation (10-bits → 1024 options)
- *I-Type*: Register-Immediate instructions
  - 1 input and 1 output register operands
  - funct3 (3 bits → 8 options) identifies the specific operation
  - 12 bits for the immediate

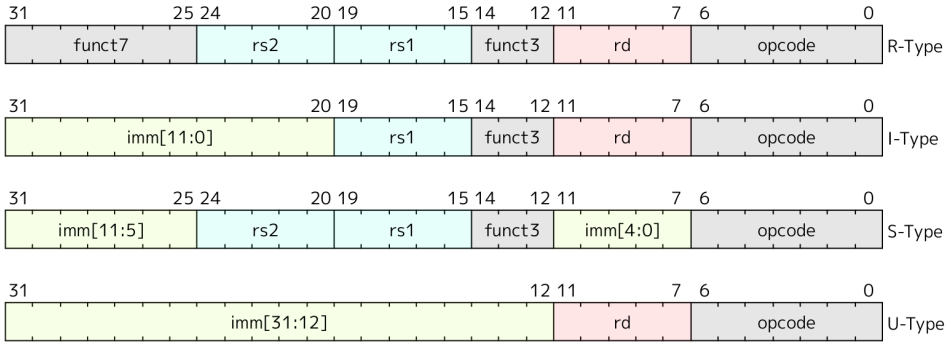
Instruction Encoding



From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

- *S-Type*: Store instructions
  - 2 input register operands
  - funct3 (3 bits → 8 options) identifying the width
  - 12 bits for the immediate (always signed)
  - Immediate value is relative to rs2: address = rs2 + immediate
  - Similar to I-Type, but different layout to keep registers in standard positions
- *B-subtype*: Branch Instructions
  - (Near) identical layout → Different layout only for immediate bits
  - Immediate value is relative to PC: target = PC + immediate
  - Immediate encodes bits 12:1 instead of bits 11:0.
    - Instruction aligned at 4 bytes for RV32I but at 2 bytes for other variants
    - Branch target has to be multiple of 2 → No need to encode bit 0 of target

Instruction Encoding



From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

- *U-Type*: Long address instructions
  - 1 destination register
  - 20 bits for the immediate
  - U-Type[31:12] are Immediate[31:12], other bits ← 0
- *J-subtype*: Jump and Link
  - Same layout
  - J-Type[31:12] are Immediate[20:1], other bits ← 0
  - target = PC + immediate
    - target, PC, and immediate are aligned at two bytes

Example – add x1, x2, x3

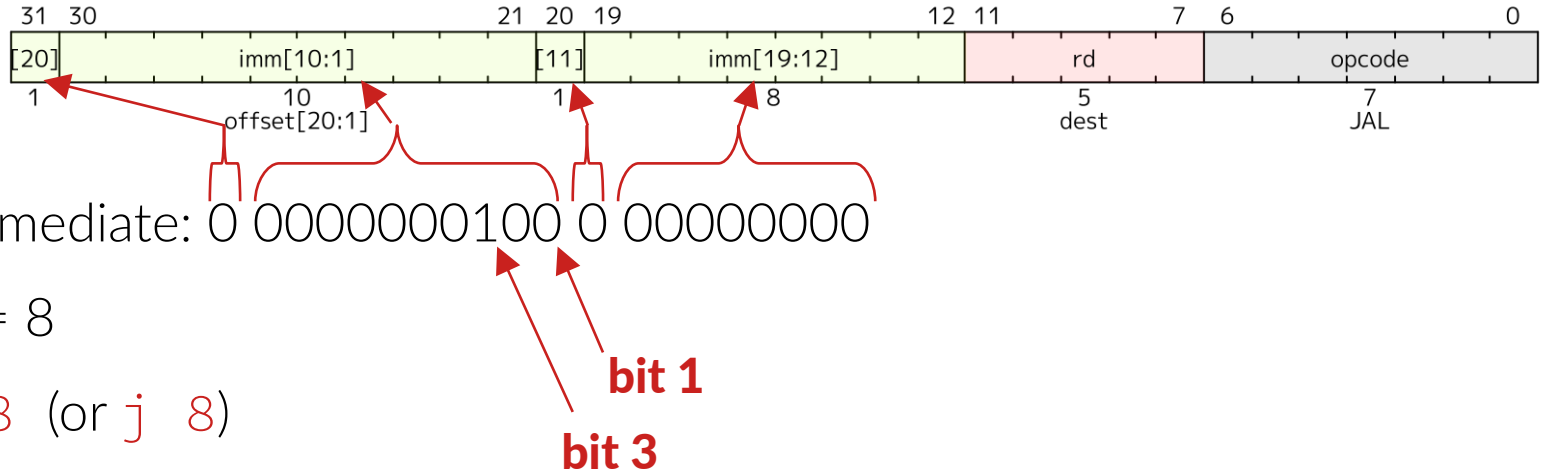
- add → OP opcode = 0110011_2
- x1 → rd = 00001_2
- x2 → rs1 = 00010_2
- x3 → rs2 = 00011_2
- func3 = 000_2
- func7 = 0000000_2
- $0000000\ 00011\ 00010\ 000\ 00001\ 0110011_2$ =
  - $00\ 31\ 00\ B3_{16}$

0000000	rs2	rs1	000	rd	0110011	ADD
0100000	rs2	rs1	000	rd	0110011	SUB
0000000	rs2	rs1	001	rd	0110011	SLL
0000000	rs2	rs1	010	rd	0110011	SLT
0000000	rs2	rs1	011	rd	0110011	SLTU
0000000	rs2	rs1	100	rd	0110011	XOR

From the "The RISC-V Instruction Set Manual" Volume I, © RISC-V

Example – Decoding

- $0080006F_{16}$
  - 0000 0000 1000 0000 0000 0000 0110 1111
  - Opcode = Bits[6:0] = 1101111 → JAL
  - rd = Bits[11:7] = 0 → x0



  - Encoded immediate: 0 0000000100 0 00000000
  - immediate = 8
  - `jal x0, 8` (or `j 8`)

Summary

- Four base instruction encodings
  - Plus two subtypes
- Simple & regular structure
  - Faster assembing
  - Faster decoding

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

Immediate Values Revisited

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Encoding immediate values has implications on how the instruction set works

Immediate values (recap)

- Literals (aka numbers or characters)
  - `li x1, 25`
- Labels (symbolic names corresponding to memory addresses)
  - The assembler will replace the label with the right immediate value
  -
```
        beq x1, x0, skip ; skip is the address of the instruction after the label
        lw x2, variable  ; variable will be replaced with the address of the defw
skip
        …
variable   defw 100
```
- Symbols (symbolic name for a constant)
  -
```
max_val   equ 0xFF
…
          li x1, max_val  ; Using max_val is replaced with 0xFF
```

Immediate values

- All of them transformed into numbers
- The number is encoded in the immediate field of the instruction

Immediate values

- Two groups of encoded immediate values
  - I/S/B-Type → 12 bits
    - Almost all instructions with immediate values
  - U/J-type → 20 bits
    - lui, auipc, and jal

I-Type immediate values

- 12-bit *constant* arguments can count from *-2,048 to 2,047* or from *0 to 4,096*
  - A tiny part of the space possible 32-bit values
  - e.g., `addi x1, x1, 1000` is valid, but
  - `andi x1, x1, 0x00000FF00` is invalid!
  - Rationale: Small values are more common
  - Trade-off between using more instruction bits and allowing larger values

B-type immediate values

- 12-bit *branch* offsets are interpreted as
  - *Relative to the current PC*
  - *Signed (can branch backward or forward)*
  - *Multiplied by 2* (so they go from -4,096 to 4,095)
  - This is a very narrow range, but most conditional branches are local, i.e. the same function

Load/Store immediates values

- 12-bit offsets
  - *-2,048 to 2,047* and
  - *Relative to the value in the associated register*
    - e.g. lw x1, 1000[x2] → address = x2 + 1000
  - Limited range on its own
  - Wider range by adjusting the address in the register

J-Type (`jal`) immediate values

- 20-bit offsets: much wider range
  - Long unconditional jumps in completely different parts of the program
  - PC-relative

U-Type immediate values

- `lui, auipc`
  - Load 20-bit immediate in upper 20 bits of a register
  - *Load exactly what doesn't fit in a normal immediate argument!*
  - Will become useful soon

Immediates

- Do you need to worry about this?
  - A little bit
  - If the immediate value is invalid, the assembler will produce an error
- With pseudo-instructions, the assembler will construct large immediate values piecewise, if that's necessary, using *U-type instructions*
  - *Next video*

Summary

- 12-bit immediates
  - Small values
  - Local conditional branches
  - Accessing data from a specific memory region
- 20-bit immediates
  - Long unconditional jumps
  - Combined with 12-bit immediates to construct 32-bit addresses in two steps

MANCHESTER 1824
The University of Manchester

COMP15111 Fundamentals of Computer Architecture

RISC-V Leftovers

Pavlos Petoumenos
pavlos.petoumenos@manchester.ac.uk

Overview

- Wrap up some loose ends about the RISC-V instruction set
- Useful assembler directives
- Our “OS”

Environment Calls (`ecall`)

- Real systems
  - User code executes with restrictions
  - System code (e.g. OS) executes unrestricted
  - Memory management, Input-Output (I/O), anything that causes inter-process interference
- How to execute privileged operations from user code?
  - System code implements them securely
  - Provides a narrow and well-defined interface
  - The hardware provides at least one way to call this code with higher privileges
- `int` in x86, `svc` in ARM, `ecall` in RISC-V

Environment Calls (`ecall`)

You don't care about this part… yet

- Basic mechanism (more later and in COMP27112)
  - Execute ecall
    - Privilege level changes to "Machine"
    - PC points to a specific location in memory where system code is
    - System code reads the registers to choose privileged operation
    - Gets input arguments from registers (optional)
    - Does its thing
    - Puts return values is registers (optional)
    - Executes mret
  - Execution of user code continues

Environment Calls (`ecall`)

- System code => Treat it as magic
- What you need to know
  - RISC-V only specifies what the ecall instruction itself does
  - What the code handling the ecall does is system software specific
  - Different RISC-V OSes/Supervisors will specify different ways to pass information

Environment Calls (`ecall`)

- Bennett has a fake "OS" providing I/O functionality
  - Operation number is expected in x17
  - Input/output argument is expected in x10

Op (x17)	Name	Input (x10)	Output (x10)	Description
0	WriteChar	ASCII char in lower 8-bits	-	Print a single ASCII character to the console
1	ReadChar		ASCII char in lower 8-bits	Read a single ASCII character from keyboard
2	WriteString	Start address of string		Print a null-terminated ASCII string to console
3	WriteDecimal	32-bit value to print		Print a single number in a decimal representation
5	Stop			Halt execution

Pseudo-Instructions (again)

- Not part of the instruction set
  - Too complicated to be a single instruction
  - Too trivial too be an extra instruction
- But you can use them in assembly
- The assembler will translate them
  - Using one or more real instructions
  - Using (perhaps) extra registers
  - Using (perhaps) extra load/stores
  - Most of the time your don't care
  - The assembler will work out the minimal code needed
  - Disassembled machine code in Bennett might be unexpected (try it!)

Large Immediate pseudo-instructions

```
• li rd, imm        (short) → addi rd, x0, imm
                     (long) → lui rd, (20 upper bits of imm)
                              addi rd, rd, (12 lower bits of imm)

• la rd, label      (short) → addi rd, x0, label
                     (long) → auipc rd, (20 upper bits of label - pc)
                              addi rd, rd, (12 lower bits of label - pc)

• lw rd, label      (short) → lw rd, label[x0]
                     (long) → auipc rd, (20 upper bits of label - pc)
                              lw rd, (12 lower bits of label - pc)[rd]

• sw rs, label, td (short) → sw rs, label[x0]
                     (long) → auipc td, (20 upper bits of label - pc)
                              sw rs, (12 lower bits of label - pc)[td]
```

Assembler Directives

- Not instructions
- Do not produce code
- Tell the assembler to do something
- Seen already
  - `symbol equ value` → `symbol` becomes an alias for "`value`"
  - `defw` num → reserves a word of store and puts the value "num" in it
  - `label defw` num → `label` is the address of the following word ("num")

Assembler Directives

- `squares defw 0, 1, 4, 9, 16, 25` → `defw` can reserve multiple words, "`squares`" is the address of the first item
- `defb 255` → `defb` reserves bytes instead of words
- `defb 'H'` → `defb` reserves a byte containing the ASCII code for 'H'
- `str defb 'H', 'e', 'l', 'l', 'o', 0` →Translates the characters to ASCII and stores them in consecutive bytes: a string!
  - `str defb "Hello", 0`
- `arr defs 16` → Reserves 16 bytes
- `arr defs 16, 5` → Reserves 16 bytes and initialises them with 5

Assembler Directives

- `align` → leave a few bytes blank to make the next item start on a word boundary (multiple of four)
  - Code needs to be aligned on words
  - Data not necessarily aligned (`defb`)
  - `align` fixes alignment when we go from data to code
- `origin addr` → The following item (data or code) starts on address `addr`
- `entry` → Following line is the starting address of the program (default is 0)

Summary

- Calling the OS → ecall
  - Print characters, strings
  - Read characters
  - Stop
- Using 32-bit immediates
  - li, la, lw, sw pseudo-instructions
- Allocating words, bytes, strings → defw, defb
- Controlling the assembler → equ, origin, align, entry